BÖLÜM 4: ASENKRON UFUKLAR — OLAY ODAKLI MIMARI

💣 9 dk • DERS 13

Parça 13: Senkron Cehennemden Asenkron Huzura: Kuyruklar, Konular ve Akışlar

Yavaş bir üçüncü parti SMS sağlayıcısının ödeme akışınızı felç etmesini engellemenin yolları.

🔥 SAHADAN YAŞANMIŞ KRİZ HİKAYESİ

Bir havayolu biletleme sisteminde 'Bileti Al' butonuna basıldığında tek bir senkron HTTP isteği tetikleniyordu. Backend bu tek istek içinde sırayla: 1) Banka Sanal Posu, 2) Koltuk Rezervasyonu, 3) Sadakat Puanı Kontrolü, 4) Dolandırıcılık ML Analizi, 5) SMS Gateway ve 6) PDF Bilet Oluşturma servislerini senkron olarak çağırıyordu. Yılbaşı gecesi SMS sağlayıcısı aşırı yoğunluktan çöktü ve her isteğe 30 saniye sonra zaman aşımı (timeout) dönmeye başladı. Bütün web sunucusu thread'leri SMS cevabını beklerken kilitlendi. 4 dakika içinde şirketin tüm sunucuları yanıt veremez hale geldi; kullanıcılar bilet almayı bırakın, ana sayfada uçuş araması bile yapamadı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Senkron istek-yanıt (request-response) mekanizması zamansal bir bağımlılık tuzağıdır: Sistem B o anda ayakta, hızlı ve erişilebilir değilse Sistem A işini tamamlayamaz. Asenkron olay odaklı mimaride ise üretici sadece olayı duyurur ('SiparisAlindi') ve kullanıcıya anında 202 Accepted döner. Üretici o olayı kimin dinlediğini bilmez. E-posta veya SMS servisi o an bakımda olsa bile mesajlar güvenli bir kuyrukta bekler ve servis açıldığında tek bir müşteri bile kaybetmeden işlenir.

Temel Mimari İlkeler ve Zihinsel Modeller

Mesajlaşma Yelpazesi: Kuyruk vs. Konu vs. Akış

- **Noktadan Noktaya Kuyruk (RabbitMQ, SQS):** İş kuyruğu. Her mesajı çalışanlardan yalnızca BİRİ işler (Örn: Bir resmi boyutlandırmak).
- **Yayınla-Abone Ol Konusu (SNS, Pub/Sub):** Fan-out modeli. Bir olay birden fazla aboneye kopyalanarak dağıtılır (Örn: Sipariş bildirimleri).
- **Olay Akışı / Dağıtık Günlük (Kafka, Kinesis):** Diskte günlerce veya aylarca saklanan sıralı günlük (commit log). Tüketiciler kendi hızlarında okur, geçmişe dönüp tekrar oynatabilir.

Komut (Command) vs. Olay (Event) Ayrımı

- **Komut ('FaturaEpostasiGonder'):** Doğrudan niyet belirtir. Üretici ne yapılması gerektiğini ve kimin yapacağını bilir.
- **Olay ('FaturaKesildi'):** Geçmişte gerçekleşmiş değiştirilemez bir olgudur. Üretici ne olduğunu söyler; kimin nasıl tepki vereceğine karışmaz.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Ölümcül Senkron Zincir:** A servisi B'yi çağırır, B C'yi, C D'yi. Her servisin %99 erişilebilirliği varsa, 4 servislik zincirin toplam başarı şansı sadece %96'dır (0.99^4).
- **Akıllı Aracı, Aptal Tüketici:** İş mantığını Enterprise Service Bus (ESB) gibi kuyruk araçlarının içine gömmek yerine aracıyı aptal, uç noktaları akıllı tutmamak.

- **Sonsuz Yeniden Deneme (Poison Pill):** Bozuk bir mesajın fırlattığı hatayı yakalayıp Dead Letter Queue'ya (DLQ) atmadan 1 milyon kez tekrar denemek ve işleyicileri kilitlemek.

> **🎯 MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Kullanıcıya anında dönen HTTP uçlarını tara: Kullanıcının ekranda hemen görmesi gerekmeyen tüm işleri (e-posta, log, rapor) asenkron kuyruğa taşı.
> - **2. Adım:** Sistemindeki her kuyruğa maksimum 3 denemeden sonra devreye giren bir Dead Letter Queue (DLQ) tanımla.
> - **3. Adım:** Olay adlandırmalarında her zaman geçmiş zaman kullan: `UserRegistered`, `PaymentCaptured`.

Mimari Çıkarımlar ve Hile Tablosu

- Zamansal ayrıştırma (temporal decoupling), sistem erişilebilirliğine verebileceğiniz en büyük hediyedir.
- Olaylarla geçmiş olguları duyurun; senkron zincirlerle başkalarına emir yağdırmayın.
- Dead Letter Queue'su olmayan her kuyruk, tek bir bozuk mesajla tüm çalışanlarınızı zehirleyebilir.

> **🎙️ UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Uygulamanızdaki ana kayıt veya satın alma akışına bakın: Kullanıcıya yanıt dönmeden önce arka planda senkron çağrılan kaç servis var? Hangileri asenkron kuyruğa taşınabilir?